

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

DIPARTIMENTO DI INGEGNERIA ELETTRICA E TECNOLOGIE
DELL'INFORMAZIONE

CORSO DI LAUREA TRIENNALE IN INFORMATICA

A FANCY L^AT_EX TEMPLATE FOR YOUR THESIS
AT UNINA

Relatori

Megadir. Prof. Guidobaldo Maria

RICCARDELLI

Duca Conte Francesco BARAMBANI

Baronessa Filiguelli DE BONCHAMPS

Correlatore

Megadir. Clam. Duca Conte Pier

Carlo ing. SEMENZARA

Candidato

Marco FESTA

N86004979

Anno Accademico 2025–2026

Sommario

Il presente lavoro di tesi illustra le attività di ricerca e sviluppo condotte durante il periodo di tirocinio presso **InfoCube**, focalizzandosi sull'estrazione, strutturazione e interrogazione semantica avanzata di documenti legali e normativi. Il progetto si inserisce nel contesto più ampio di **LUDOVICA** (*Language Understanding for Document Optimization and Validation In Civil Administration*), un motore generativo basato sull'Intelligenza Artificiale volto a supportare la Pubblica Amministrazione Locale nell'elaborazione e stesura di atti amministrativi.

Il problema tecnico principale affrontato in questo lavoro è la trasformazione di documenti PDF complessi in una base di conoscenza solida e interrogabile dalle macchine (**Knowledge Graph**). A tale scopo, la rigida gerarchia tipica degli atti normativi (Titoli, Capi, Articoli, Commi) è stata modellata e salvata all'interno del database a grafo **Neo4j**[1], permettendo di preservare fedelmente le relazioni strutturali e concettuali dei testi originali.

Per abilitare il recupero intelligente e contestuale delle informazioni, è stato implementato un sistema di **Semantic Search**. Questa fase ha richiesto il testing di molteplici **modelli di embedding**, allo scopo di individuare la soluzione più performante per la vettorializzazione del complesso gergo giuridico italiano. Al fine di validare le scelte architetturali, è stato inoltre introdotto e impiegato il database vettoriale **Qdrant**[2] a scopo puramente sperimentale. Ciò ha permesso di condurre un'analisi comparativa diretta tra le performance di un approccio di ricerca puramente vettoriale e i vantaggi offerti dalla navigazione topologica nel grafo.

SCRIVERE QUI I RISULTATI OTTENUTI, LE CONCLUSIONI E EVENTUALI PROSPETTIVE FUTURE.

Indice

1	Introduzione	1
1.1	Definizione del problema	1
1.2	Obiettivi del lavoro di tesi	2
1.3	Descrizione dell'approccio	2
1.4	Confronto con tecniche esistenti	2
1.5	Struttura della tesi	3
2	Tecnologie utilizzate	4
2.1	Linguaggi di Programmazione	4
2.2	Strumenti per l'estrazione del Testo	4
2.3	Scelta dei Database	6
2.4	LLM e Modelli di Embedding	6
2.5	Deployment e Containerizzazione	8
3	Architettura del Sistema	9
4	Implementazione Software	11
4.1	Estrazione dei Documenti	11
4.1.1	Open Data Loader e la Gestione dei Tagged PDF	11
4.1.2	Docling e l'Integrazione dei Motori OCR	12
4.1.3	Progettazione del Custom Triage Processor	13
4.2	Strutturazione dei Documenti tramite LLM	15
4.2.1	Il Modello e la Strategia di Generazione	15
4.2.2	Generazione della Struttura Gerarchica	15
4.2.3	Estrazione delle Relazioni Semantiche	16
4.3	Importazione dati nei Database	17
4.3.1	I Modelli di Dominio	17
4.3.2	Disaccoppiamento della Persistenza: Lo Strategy Pattern	18
4.3.3	Modalità di Accesso ai Dati: Il Repository Pattern	19
4.3.4	Dettagli Implementativi dell'Ingestion su Neo4j	19
4.3.5	Dettagli Implementativi dell'Ingestion su Qdrant	20
4.3.6	Risultati e Metriche	21
4.4	Creazione degli Embedding	21

4.4.1	Astrazione e Gestione dei Modelli di Embedding	22
4.4.2	Integrazione con i Database	23
5	Ricerca Semantica	24
5.1	Introduzione alla Ricerca Semantica	24
5.2	Setup Sperimentale	24
5.2.1	Definizione della soglia di ricerca: il parametro Top-K	25
5.2.2	Allineamento delle strategie di ricerca: ANN vs Exact k-NN	25
5.3	Metriche di valutazione	25
5.4	Risultati Sperimentali	26
5.4.1	Precisione a Confronto (HIT@K)	26
5.4.2	Analisi dei Tempi e Latenze	26
5.5	Analisi Comparativa e Conclusioni	27
5.6	Embedding di Neo4j con relazioni	28
	Bibliografia	29

Elenco delle figure

3.1	Architettura del sistema	10
4.1	Flowchart del Custom Triage Processor	14
4.2	Class Diagram dei modelli di dominio	18

Elenco delle tabelle

4.1	Confronto metriche e tempi tra Neo4j e Qdrant.	21
5.1	Risultati di Precisione (identici per Neo4j e Qdrant)	26
5.2	Latenza Media Aggregata di Ricerca Vettoriale	27

-1-

Introduzione

CONTENTS: 1.1 Definizione del problema. 1.2 Obiettivi del lavoro di tesi. 1.3 Descrizione dell'approccio. 1.4 Confronto con tecniche esistenti. 1.5 Struttura della tesi.

1.1 Definizione del problema

La pubblica amministrazione locale italiana fa quotidianamente affidamento su una vasta e complessa mole di testi normativi, atti e delibere. L'elaborazione automatica di tali documenti è tuttavia ostacolata da diverse sfide intrinseche: i testi sono spesso archiviati in formati non strutturati (come i PDF), presentano difformità di impaginazione, impiegano un linguaggio burocratico peculiare e si articolano in una rigida e profonda struttura gerarchica. Quando si tenta di automatizzare la ricerca e la stesura di nuovi atti attraverso sistemi basati su Intelligenza Artificiale Generativa, l'approccio classico - basato unicamente sul recupero esplorativo di porzioni di testo destrutturato (frammenti di testo o "chunk") - si rivela insufficiente, portando inevitabilmente alla potenziale perdita del contesto legale e allo smarrimento delle relazioni gerarchiche originarie dell'atto documentale.

È in questo scenario che si inserisce il lavoro di tesi, svolto nell'ambito delle attività di tirocinio presso **InfoCube** per contribuire allo sviluppo del progetto **LUDOVICA**, il cui obiettivo finale è fornire ai funzionari della PA un motore generativo affidabile, capace di recuperare gli atti pregressi pertinenti e generare template documentali pronti all'uso. Affinché le risposte del sistema siano rigorose e verificabili è emersa l'esigenza di adottare un'architettura dati che introduca una solida topologia informativa.

Il problema tecnico fondamentale affrontato consiste quindi nella necessità di **convertire i documenti normativi complessi in una base di conoscenza interrogabile** (*Knowledge Graph*) che conservi fedelmente la struttura semantica e gerarchica originale (Titoli, Capi, Articoli).

1.2 Obiettivi del lavoro di tesi

Al fine di superare i limiti esposti e dotare il sistema di solidi meccanismi per l'interpretazione del linguaggio giuridico, il lavoro si è articolato nei seguenti obiettivi:

1. **Modellazione topologica e digitalizzazione:** Costruire una pipeline di inserimento per tradurre documenti legali non strutturati in un database a grafo (*Neo4j*), definendo rigorosamente la mappatura delle relazioni che legano tra loro le sezioni della gerarchia normativa.
2. **Semantic Search per il dominio giuridico:** Sviluppare il modulo di interrogazione intelligente del database e quantificare, mediante il testing di molteplici modelli di embedding, quale sia la soluzione più performante nel catturare la semantica intrinseca del linguaggio legale in lingua italiana.
3. **Analisi prestazionale degli approcci di Retrieval:** Integrare un database puramente vettoriale (*Qdrant*) per poter eseguire un **confronto diretto**. L'obiettivo è misurare la differenza qualitativa delle risposte e confrontare le prestazioni fornite da un approccio di ricerca puramente vettoriale, rispetto a un'interrogazione che sfrutti anche la navigazione topologica sul grafo.

1.3 Descrizione dell'approccio

Per raggiungere gli obiettivi prefissati, è stata ideata un'architettura di elaborazione in grado di trasformare documenti non strutturati in una rete di nodi interconnessi. L'approccio proposto si articola in una pipeline a più fasi: inizialmente, i documenti originali (principalmente in formato PDF) vengono processati tramite strumenti di parsing avanzati e OCR per estrarne accuratamente il testo e convertirlo in un formato intermedio (Markdown). Successivamente, un *Large Language Model* analizza tale testo per riconoscerne la sintassi e la suddivisione gerarchica (Titoli, Capi, Articoli, Commi). Questi elementi vengono strutturati in formato JSON e iniettati all'interno del grafo, in cui ogni entità diventa un nodo e l'appartenenza a una data sezione diventa una relazione. Successivamente, i contenuti testuali vengono vettorializzati e salvati sotto forma di *embedding*. In fase di interrogazione, il sistema non si limita a cercare la similarità matematica tra le frasi, ma naviga esplicitamente la topologia del grafo per ricostruire l'intero iter contestuale, fornendo al modello generativo un prompt documentato, ricco e strutturalmente coerente.

1.4 Confronto con tecniche esistenti

Nel panorama attuale dell'Intelligenza Artificiale applicata all'elaborazione dei documenti, le tecniche di *Information Retrieval* più diffuse si basano principalmente su *Retrieval-Augmented Generation* (RAG) di tipo vettoriale. Quest'ultima prevede tipicamente la

suddivisione del documento in frammenti testuali di lunghezza fissa (*chunking*), la loro vettorializzazione e il successivo recupero basato sulla distanza semantica rispetto alla query dell'utente.

Tuttavia, queste tecniche presentano lacune significative quando applicate al dominio giuridico della pubblica amministrazione. Un approccio RAG standard, estraendo un singolo *chunk* isolato, perde inevitabilmente il perimetro normativo: ad esempio, potrebbe recuperare il comma di una legge senza specificare a quale Capo appartenga oppure omettendo dettagli cruciali come eventuali eccezioni specificate in altri paragrafi. L'approccio ibrido basato su *Knowledge Graph* (spesso definito *GraphRAG*) si rende pertanto strettamente **necessario**. Modellando esplicitamente le relazioni tra i componenti di una legge, esso supera i limiti del chunking piatto, consentendo di restituire un contesto normativo integro e affidabile. Questo aspetto è fondamentale per abbattere drasticamente le allucinazioni tipiche degli LLM e garantire l'assoluta affidabilità formale e legale delle risposte, un requisito imprescindibile per un software destinato agli operatori della PA.

1.5 Struttura della tesi

Il presente elaborato è organizzato in cinque capitoli, di seguito brevemente introdotti.

Il **Capitolo 2 — Tecnologie utilizzate** fornisce una panoramica dello stack tecnologico adottato: dai linguaggi di programmazione agli strumenti di parsing e OCR per l'estrazione del testo, passando per i database a grafo e vettoriali (Neo4j e Qdrant), fino ai modelli di linguaggio e di embedding impiegati per la strutturazione semantica e la vettorializzazione dei contenuti normativi.

Il **Capitolo 3 — Architettura del Sistema** illustra l'architettura complessiva del sistema, descrivendo i componenti principali e il flusso di dati attraverso le diverse fasi della pipeline: dall'acquisizione dei documenti grezzi fino all'interrogazione da parte degli utenti finali.

Il **Capitolo 4 — Implementazione Software** approfondisce nel dettaglio le singole fasi della pipeline di elaborazione: l'estrazione del testo dai documenti PDF, la strutturazione gerarchica operata dai Large Language Models, l'ingestion dei dati strutturati su Neo4j e Qdrant e la creazione degli embedding per la ricerca semantica.

Il **Capitolo 5 — Ricerca Semantica Vettoriale** presenta il setup sperimentale, le metriche di valutazione adottate e i risultati ottenuti con l'approccio di ricerca puramente vettoriale sia su Neo4j che su Qdrant.

Il **Capitolo 6 — Ricerca Semantica con Grafo** analizza i risultati del confronto tra l'approccio basato sul grafo e quello puramente vettoriale, evidenziando le differenze in termini di performance ed efficacia nella restituzione del contesto normativo.

–2–

Tecnologie utilizzate

Per far fronte alle sfide di estrazione, elaborazione, archiviazione e interrogazione dei documenti normativi, è stato impiegato uno stack tecnologico che combina strumenti di parsing avanzato, modelli di LLM ed embedding e database specializzati. Di seguito vengono presentate le tecnologie adottate, illustrando per ciascuna le motivazioni che ne hanno guidato la scelta.

2.1 Linguaggi di Programmazione

- **Python**[3]: È stato il linguaggio cardine per la costruzione dell'intera pipeline del progetto. La scelta è ricaduta su Python per diverse ragioni:
 - **Ecosistema dedicato ai modelli linguistici:** Python dispone di librerie mature e ampiamente supportate come `sentence-transformers` e `huggingface-hub`, che hanno permesso di integrare agevolmente i modelli di embedding e i workflow di Retrieval-Augmented Generation senza dover reimplementare componenti di basso livello.
 - **Integrazione con i database:** I driver ufficiali per Neo4j e Qdrant offrono API Python native, ben documentate e performanti, eliminando la necessità di adattatori o wrapper di terze parti.

2.2 Strumenti per l'estrazione del Testo

La fase di estrazione ha dovuto confrontarsi con documenti PDF eterogenei: alcuni nativamente digitali e ben strutturati, altri derivanti da scansioni cartacee di qualità variabile. Per questo motivo è stata adottata una combinazione di strumenti complementari.

- **Open Data Loader**[4]: Libreria open-source per l'estrazione e il caricamento di dati da fonti eterogenee. È stata selezionata per tre caratteristiche fondamentali:

- **Determinismo:** A differenza di approcci basati su modelli probabilistici, Open Data Loader estrae il testo in modo deterministico, garantendo riproducibilità e coerenza dei risultati — un requisito indispensabile nel dominio legale, dove ogni modifica del testo estratto tra un'esecuzione e l'altra minerebbe la fiducia nel sistema.
- **Conversione diretta in Markdown:** Il Markdown si rivela un formato testuale ottimale per i LLM. La sua sintassi leggera preserva fedelmente la struttura logica del documento originale, fornendo al modello un contesto già strutturato che riduce significativamente gli errori nella successiva fase di parsing gerarchico.
- **Docling**[5]: Libreria specializzata nell'analisi del layout documentale e nell'estrazione di contenuti strutturati. È stata integrata come supporto complementare a Open Data Loader per la gestione di PDF più complessi: supporta documenti multi-colonna, tabelle e layout non lineari, frequenti negli atti della pubblica amministrazione.
- **RapidOCR**[6]: Motore di *Optical Character Recognition* impiegato per digitalizzare i documenti derivanti da scansioni visive. La scelta è stata guidata da un'analisi comparativa tra diversi motori OCR:
 1. **EasyOCR:** Configurato di default all'interno di Docling, adotta modelli basati su Deep Learning altamente accurati ma estremamente esigenti in termini computazionali. Anche isolando l'elaborazione sulla singola pagina, il consumo di RAM superava costantemente la soglia critica dei 4 GB, rendendone impossibile l'utilizzo nell'ambiente di produzione containerizzato.
 2. **Tesseract OCR:** Si è dimostrato estremamente leggero e pienamente compatibile con i limiti di memoria imposti, scongiurando qualsiasi crash del container. Tuttavia, nel contesto specifico del linguaggio legale italiano, il motore non è stato in grado di interpretare correttamente il layout, fallendo l'estrazione del testo e restituendo output incompleti o disallineati rispetto alla struttura originale del documento.
 3. **RapidOCR:** Questo specifico modulo è stato infine selezionato per l'estrazione del testo dalle scansioni visive. RapidOCR si è rivelato il compromesso ingegneristico ideale, offrendo un perfetto bilanciamento tra l'accuratezza del riconoscimento del testo italiano e il consumo di RAM del container, che si è mantenuto costantemente sotto la soglia di guardia. A fronte di un tempo di computazione non trascurabile — quantificabile in circa 18-20 secondi per singola pagina scansionata — questo motore assicura la totale stabilità infrastrutturale dell'intera pipeline.

2.3 Scelta dei Database

La duplice natura del problema — da un lato la necessità di rappresentare relazioni gerarchiche complesse, dall'altro l'esigenza di una ricerca semantica efficiente — ha portato all'adozione di due database complementari, ciascuno ottimizzato per uno specifico paradigma di interrogazione.

- **Neo4j**: Il database a grafo scelto come storage principale per la base di conoscenza normativa. La decisione di adottare un database a grafo anziché un tradizionale database relazionale è motivata da considerazioni sia modellistiche che prestazionali:
 - **Modellazione naturale della gerarchia normativa**: La struttura di un atto normativo — articolata in Titoli, Capi, Articoli e Commi — è intrinsecamente un grafo, in cui ogni elemento è un nodo e le relazioni di appartenenza sono archi. Neo4j consente di mappare questa topologia in modo diretto, senza le complessità delle JOIN ricorsive tipiche dei database relazionali.
 - **Flessibilità dello schema**: Il modello a grafo non impone uno schema rigido, permettendo di arricchire progressivamente la base di conoscenza con nuove tipologie di nodi e relazioni senza dover migrare lo schema esistente.
 - **Maturità e affidabilità**: Neo4j è il database a grafo più diffuso e maturo sul mercato, con una documentazione estesa e driver ufficiali per Python. Questo ha ridotto i rischi legati all'adozione di una tecnologia di nicchia in un contesto progettuale reale.
- **Cypher**[7]: Il linguaggio di interrogazione dichiarativo nativo di Neo4j, utilizzato per effettuare le query di inserimento e di ricerca topologica nel grafo.
- **Qdrant**[2]: Database vettoriale scritto in Rust, impiegato come termine di paragone sperimentale per valutare l'efficacia dell'approccio basato su grafo. La scelta è stata motivata da:
 - **Performance elevate**: Implementato in Rust, Qdrant offre operazioni di ricerca per similarità estremamente veloci anche su collezioni di grandi dimensioni.
 - **Confronto metodologico**: L'utilizzo di Qdrant ha permesso di isolare il contributo della topologia del grafo rispetto alla pura similarità semantica, quantificando il vantaggio dell'approccio GraphRAG in termini di pertinenza e completezza dei risultati.

2.4 LLM e Modelli di Embedding

Il cuore intelligente del sistema risiede nei modelli di linguaggio e di embedding, responsabili rispettivamente della strutturazione semantica dei documenti e della loro vettorializzazione per la ricerca.

- **Gemini 3.5 Flash**[8]: *Large Language Model* di Google, utilizzato tramite API per il compito cruciale della strutturazione sintattica dei documenti normativi. Le motivazioni alla base di questa scelta sono:
 - **Aderenza rigorosa allo schema JSON**: Gemini 3.5 Flash ha dimostrato un'eccellente capacità di produrre output JSON conformi a uno schema predefinito, un requisito fondamentale per garantire che la gerarchia normativa (Titoli, Capi, Articoli, Commi) fosse rappresentata in modo consistente e automaticamente processabile dalle fasi successive della pipeline.
 - **Comprensione della lingua italiana**: Il modello ha mostrato una solida competenza nell'elaborazione del linguaggio burocratico italiano, riconoscendo correttamente la terminologia giuridica e le espressioni formulari tipiche degli atti amministrativi.
 - **Rapporto costo/qualità**: Rispetto a modelli più potenti ma significativamente più costosi (come GPT-4 o Claude), Gemini 3.5 Flash ha offerto un compromesso ottimale tra qualità dell'output strutturato e costo per token, aspetto rilevante considerando l'elaborazione batch di centinaia di documenti.
- **Modelli di Embedding**: La vettorializzazione del testo normativo rappresenta un elemento critico per la qualità della ricerca semantica. Sono stati testati molteplici modelli per identificare la soluzione più efficace nel dominio giuridico italiano. I test più approfonditi hanno coinvolto i seguenti modelli:
 - `intfloat/multilingual-e5-large`[9]: Modello multilingue di grandi dimensioni, scelto per la sua comprovata efficacia su benchmark multilingue. Supporta oltre 100 lingue e utilizza un approccio di embedding basato su prompt-template, che consente di adattare la rappresentazione vettoriale al tipo di task (query vs. documento).
 - `google/gemini-embedding-001`[10]: Modello proprietario di Google, testato per valutare i benefici dell'integrazione nativa con l'ecosistema Gemini e per la sua capacità di produrre embedding di alta qualità ottimizzati per task di retrieval.
 - `google/embeddinggemma-300m`[11]: Modello compatto da 300 milioni di parametri basato sull'architettura Gemma. È stato incluso nella sperimentazione per valutare il trade-off tra dimensione del modello e qualità degli embedding, particolarmente rilevante in scenari di deployment con risorse limitate.
 - `BAAI/bge-m3`[12]: Modello stato dell'arte sviluppato da BAAI, specificamente progettato per supportare embedding multilingue, multi-funzionali (dense, sparse, multi-vector) e multi-granulari. La sua capacità di gestire testi lunghi fino a 8192 token lo rende particolarmente adatto ai documenti normativi, dove i commi possono estendersi per diverse centinaia di parole.

La valutazione si è basata sulla capacità di ciascun modello di recuperare i commi pertinenti a partire da query in linguaggio naturale, misurata attraverso metriche standard di Information Retrieval quali *Precision@k*, *Recall@k* e *Mean Reciprocal Rank* (MRR).

2.5 Deployment e Containerizzazione

- **Docker**[13]: L'intero ecosistema è stato modularizzato attraverso lo sviluppo di container isolati e specifici per ciascun componente della pipeline. Questa scelta architetturale è stata dettata da esigenze sia tecniche che operative:
 - **Isolamento delle dipendenze:** Ogni fase della pipeline (estrazione, strutturazione, ingestione, embedding) presenta dipendenze software potenzialmente conflittuali. Docker ha permesso di incapsulare ciascuna in un ambiente dedicato, eliminando i conflitti di versione tra librerie e garantendo la riproducibilità dell'ambiente di esecuzione su qualsiasi macchina.
 - **Controllo delle risorse:** I task più onerosi, in particolare l'OCR con RapidOCR e la generazione degli embedding, sono soggetti a picchi di consumo di CPU e RAM. Docker consente di imporre limiti precisi sulle risorse allocate a ciascun container, prevenendo fenomeni di *resource starvation* che potrebbero degradare le prestazioni dell'intero sistema.
 - **Modularità e manutenibilità:** La separazione in container indipendenti ha permesso di sviluppare, testare e aggiornare ciascun modulo della pipeline in isolamento, semplificando la manutenzione e facilitando l'onboarding di nuovi sviluppatori nel progetto LUDOVICA.
 - **Scalabilità orizzontale:** L'architettura a container costituisce un prerequisito naturale per l'orchestrazione tramite Kubernetes o Docker Swarm, aprendo la strada a futuri scenari di deployment distribuito in cui diverse istanze della pipeline possano operare concorrentemente su lotti distinti di documenti.

–3–

Architettura del Sistema

In questo capitolo viene presentata l'architettura complessiva del sistema sviluppato, illustrando i componenti principali e il flusso di dati attraverso le diverse fasi del processo. L'obiettivo è fornire una panoramica chiara e dettagliata delle scelte progettuali adottate per affrontare le sfide tecniche del progetto, tracciando il percorso logico che i dati seguono dal momento in cui vengono acquisiti fino alla fase di interrogazione e utilizzo da parte degli utenti finali.

La Figura 3.1 fornisce un diagramma riassuntivo di tale architettura, evidenziando i moduli principali e le interazioni tra di essi. Come illustrato nello schema, il sistema è concepito come una pipeline sequenziale.

Nella prima fase, vengono applicate tecniche specifiche per il recupero dei testi grezzi dai documenti normativi, con particolare attenzione alle librerie impiegate per il parsing e alle strategie di pulizia dei dati, fondamentali per mitigare rumore e inconsistenze strutturali.

Una volta estratto, il testo necessita di essere organizzato secondo le logiche giuridiche del documento. A tale scopo, la fase di strutturazione si avvale di Large Language Models (LLM) per la trasformazione del testo grezzo in un formato analizzabile. Il fulcro di questo passaggio è la definizione e l'utilizzo di uno schema JSON rigoroso, progettato appositamente per rappresentare e mantenere intatta la complessa gerarchia degli atti normativi, come la suddivisione in Titoli, Capi, Articoli e Commi.

I dati validati e strutturati vengono successivamente indirizzati verso i sistemi di archiviazione. Come mostrato in Figura 3.1, l'architettura prevede l'impiego di due diverse tecnologie di storage, popolate in parallelo per consentire un confronto architetturale e prestazionale:

- Il **database a grafo (Neo4j)** è deputato alla modellazione e memorizzazione delle informazioni strutturali e semantiche. Qui, i dati vengono inseriti prestando particolare attenzione alla rappresentazione dei metadati e delle relazioni gerarchiche e sequenziali che intercorrono tra le varie parti del documento normativo. Inoltre, il database ospita anche le rappresentazioni vettoriali del testo (embedding) per abilitare la ricerca semantica all'interno del grafo stesso.

- Il **database vettoriale (Qdrant)** viene anch'esso popolato con le medesime rappresentazioni vettoriali e i relativi dati strutturati essenziali. Questa archiviazione parallela è mirata a ottimizzare la ricerca per similarità in un ambiente vettoriale puro, fungendo così da termine di paragone fondamentale per la successiva fase di benchmarking tra i due approcci.

L'infrastruttura di storage fa da fondamento al modulo di implementazione della ricerca semantica. Questo componente descrive l'approccio adottato per permettere agli utenti di interrogare la base di conoscenza, includendo le motivazioni dietro la scelta di specifici modelli di embedding e la definizione delle metriche utilizzate per valutarne l'accuratezza.

Infine, per validare le scelte architetturali proposte, il sistema include un processo di benchmarking. Questo confronto analitico è stato condotto per misurare le prestazioni di ricerca, evidenziando le differenze in termini di performance ed efficacia tra l'approccio basato sul database a grafo e quello supportato dal database vettoriale.

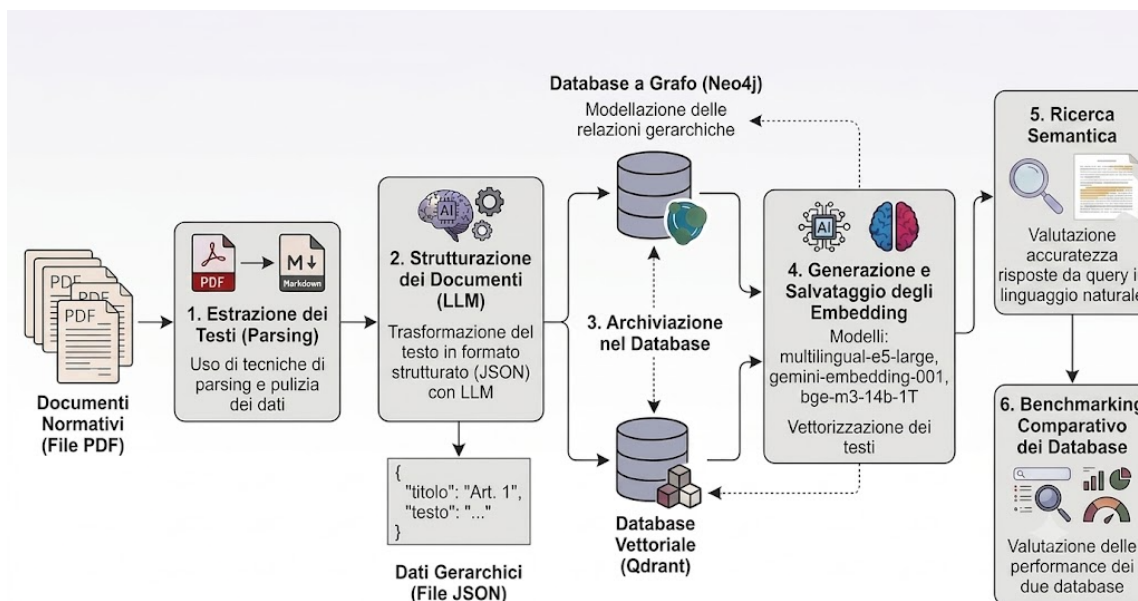


Figura 3.1: Architettura del sistema

–4–

Implementazione Software

CONTENTS: 4.1 Estrazione dei Documenti. 4.2 Strutturazione dei Documenti tramite LLM. 4.3 Importazione dati nei Database. 4.4 Creazione degli Embedding.

In questo capitolo verranno analizzate nel dettaglio le singole fasi della pipeline di elaborazione, partendo dalle strategie di estrazione del testo grezzo, passando per la strutturazione semantica operata dai Large Language Models, fino ad arrivare alle logiche di ingestione e interrogazione sui database

4.1 Estrazione dei Documenti

La fase iniziale della pipeline prevede l'estrazione del testo dai documenti normativi in formato PDF.

Nelle prime fasi di sviluppo del progetto, l'intero documento PDF veniva passato direttamente al Large Language Model (Gemini), demandando a quest'ultimo sia il compito di estrazione del testo grezzo sia quello di strutturazione sintattica. Questo approccio presentava forti criticità, esponeva il sistema al rischio di allucinazioni ed era inefficiente nel parsing di strutture complesse. Per superare tali limitazioni, la pipeline è stata modificata separando nettamente l'estrazione del testo dalla sua successiva strutturazione.

4.1.1 Open Data Loader e la Gestione dei Tagged PDF

Per l'estrazione del testo da documenti digitali nativi è stata adottata la libreria open-source *Open Data Loader*. Questo strumento converte localmente i file PDF in formati ottimizzati per i LLM, specificamente Markdown e JSON. Il vantaggio di questa soluzione risiede nella sua natura deterministica: a parità di input viene garantito il medesimo output, azzerando le variabilità tipiche dei modelli generativi.

Durante l'analisi dei documenti legali, una delle criticità maggiori riguarda i ritorni a capo spuri, generati per ragioni puramente stilistiche (ad esempio, il raggiungimento del

marginale della pagina). Se non gestiti correttamente, questi si traducono in doppi ritorni a capo nel testo Markdown finale, frammentando artificialmente i periodi e compromettendo la qualità della successiva fase di chunking. Per risolvere questa criticità alla radice, l'estrazione è stata configurata abilitando il parametro specifico `use_struct_tree=True`.

Quando questo parametro è attivo, la libreria modifica il suo approccio di lettura, verificando innanzitutto se il file analizzato appartiene alla categoria dei *Tagged PDF* [14]. A differenza dei PDF standard, che definiscono esclusivamente il layout bidimensionale e visivo, i Tagged PDF incorporano un livello gerarchico logico e invisibile: il *Logical Structure Tree*. Questo albero organizza il contenuto tramite elementi strutturali del tutto simili ai tag HTML (come `<H1>`, `<P>`, `<Table>`, `<L>`).

Di conseguenza, abilitando `use_struct_tree=True`, *Open Data Loader* è in grado di ignorare le fallaci euristiche geometriche e navigare direttamente la struttura logica del documento. Questo meccanismo permette all'estrattore di mappare ogni porzione di testo discriminando con precisione assoluta i semplici a capo visivi dalle reali interruzioni di paragrafo, restituendo un flusso di testo continuo e fedele all'originale.

Nel contesto di questa tesi, focalizzata sugli atti della Pubblica Amministrazione Locale, questa scelta tecnica si è rivelata non solo efficace, ma strategica. Le istituzioni pubbliche sono infatti vincolate da stringenti obblighi normativi sull'accessibilità digitale, che impongono la piena fruibilità dei documenti tramite tecnologie assistive, come gli screen reader. Per rispettare tali standard legali, i documenti pubblicati devono obbligatoriamente contenere un *Logical Structure Tree* rigoroso e ben formato. Pertanto, quello che per l'ente pubblico rappresenta un adempimento normativo inderogabile, si trasforma per la nostra pipeline di estrazione in una preziosa risorsa: una fonte di dati pre-strutturata in cui le gerarchie e l'ordine di lettura sono già nativamente validati alla radice.

4.1.2 Docling e l'Integrazione dei Motori OCR

L'approccio standard basato su *Open Data Loader* mostra evidenti limiti in presenza di documenti non strutturati, privi di testo selezionabile o derivanti da scansioni visive, in quanto tale libreria non integra nativamente alcun modulo OCR. Per sopperire a questa mancanza e gestire layout documentali complessi, è stata introdotta nel sistema la libreria *Docling*.

A differenza di *Open Data Loader*, *Docling* supporta l'integrazione diretta di motori OCR, configurandosi come un sistema avanzato di parsing e analisi del layout. Questo strumento, specializzato nell'estrazione di metadati, tabelle e testo da formati eterogenei, è stato quindi affiancato a *Open Data Loader* all'interno della pipeline, intervenendo come supporto specifico per processare tutti quei documenti in cui la sola estrazione strutturale risulta impossibile o inefficace.

L'introduzione dell'OCR ha sollevato complessità non trascurabili relative alla gestione delle risorse hardware all'interno dell'architettura a container. Il container Docker deputato all'estrazione operava infatti con un vincolo stringente di memoria RAM pari a un massimo di 4 GB, per questioni di risorse hardware. Durante i primi test, l'invio massivo di interi documenti scansionati causava la saturazione istantanea della memo-

ria, provocando l'intervento del modulo del kernel Linux (*Out-Of-Memory Killer*) che terminava forzatamente il container.

Per superare il problema del consumo di RAM e prevenire i crash di sistema, è stata implementata una strategia basata sull'elaborazione pagina-per-pagina. Come discusso nel Capitolo 2 relativo alle scelte tecnologiche, a seguito di un'analisi comparativa dei motori supportati da Docling, la scelta è ricaduta su **RapidOCR**. Questo motore ha permesso di mantenere il consumo di memoria costantemente sotto la soglia di guardia dei 4 GB, garantendo la totale stabilità infrastrutturale della pipeline senza sacrificare l'accuratezza del riconoscimento sul testo legale italiano.

4.1.3 Progettazione del Custom Triage Processor

Il processore di triage nativo di Open Data Loader prevede un flusso standard in cui ogni pagina viene analizzata e smistata: le pagine semplici vengono processate tramite il processo di Open Data Loader (una JVM), mentre le pagine complesse vengono delegate al backend basato su Docling.

Tuttavia, l'applicazione con l'uso del parametro `use_struct_tree=True` introduce un bug logico nel triage di default: i documenti non espressamente *tagged* vengono forzati all'interno del processo standard di Open Data Loader, non venendo mai instradati verso il backend di Docling, escludendo così l'analisi per pagine troppo complesse oppure l'attivazione dell'OCR dove necessario.

Per risolvere questa problematica è stato progettato e implementato un **Custom Triage Processor**. Come illustrato nel diagramma di flusso in **Figura 4.1**, il componente corregge le anomalie di instradamento agendo secondo una logica a tre fasi:

- **Fase 1: Verifica preliminare della struttura.** Come si evince dal primo blocco decisionale della Figura 4.1, non appena il file PDF viene acquisito, il sistema ne analizza i metadati per verificare se è codificato come *Tagged PDF*. In caso affermativo, l'intero documento viene elaborato direttamente da Open Data Loader sfruttando l'albero strutturale nativo. Questo approccio garantisce la massima velocità di esecuzione e una pulizia ottimale del testo estratto.
- **Fase 2: Frazionamento del documento e stima della complessità.** Se il documento risulta privo di struttura (*Non-Tagged*), il flusso prevede la divisione in singole pagine indipendenti. Questa separazione è fondamentale per prevenire sovraccarichi di memoria RAM durante le elaborazioni successive. Per ciascuna pagina, il sistema calcola poi la densità dei caratteri testuali per valutarne il grado di complessità.
- **Fase 3: Triage ed elaborazione mirata della pagina.** Sulla base dell'analisi precedente, ogni singola pagina subisce il processo di smistamento intelligente mostrato nella parte inferiore della Figura 4.1, venendo indirizzata allo strumento di estrazione più adeguato:

- Se la pagina presenta un layout lineare e un'elevata densità di testo nativo, viene classificata come *Semplice* e affidata al processo di Open Data Loader.
- Se la pagina contiene elementi geometrici articolati o tabelle annidate, ma il testo rimane comunque selezionabile, viene classificata come *Complessa* e processata tramite *Docling Standard* (senza abilitare l'OCR).
- Se la pagina è priva di testo nativo (es. una fotocopia), viene classificata come *Scansione* e inviata al container specializzato di *Docling*, dove interviene il motore *RapidOCR* per il riconoscimento ottico dei caratteri.

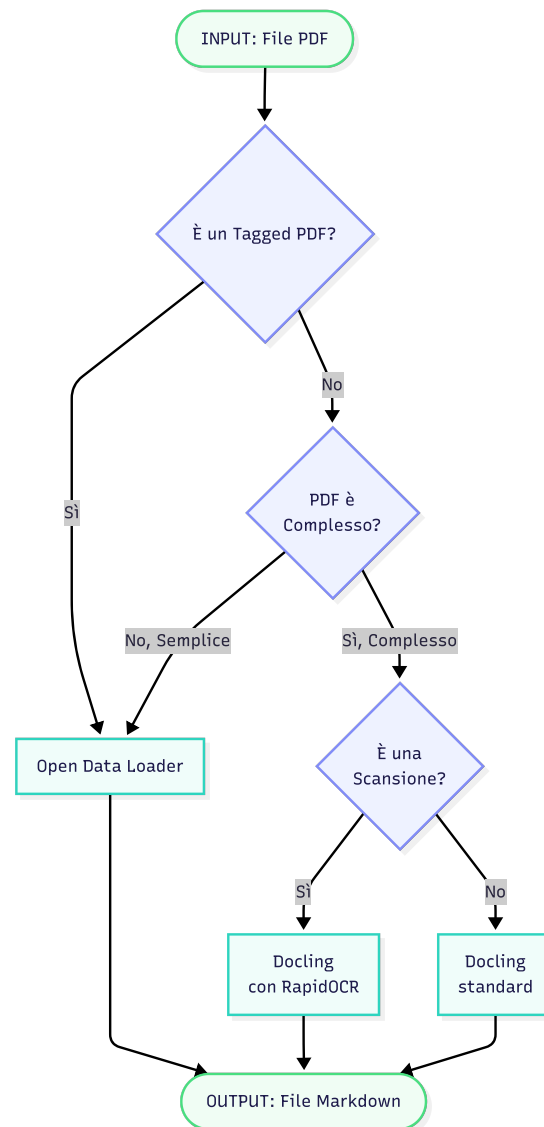


Figura 4.1: Flowchart del Custom Triage Processor

4.2 Strutturazione dei Documenti tramite LLM

Una volta ultimata la fase di estrazione del testo grezzo, illustrata nella sezione precedente, i documenti normativi si presentano sotto forma di file testuali piatti in formato Markdown. Tali file, pur contenendo l'intero testo originale, sono del tutto privi di metadati strutturali e di una formattazione gerarchica chiara. Poiché l'architettura di sistema prevede l'ingestione di questi documenti all'interno di un database a grafo (Neo4j) e in un database vettoriale (Qdrant), risulta indispensabile trasformare il testo non strutturato in una gerarchia rigorosa. L'archiviazione del solo testo grezzo, infatti, risulterebbe del tutto inadeguata e priva di utilità pratica. Una strutturazione accurata rappresenta il pre-requisito fondamentale per abilitare qualsiasi operazione avanzata di *information retrieval*, garantendo che i dati possano essere interrogati in modo efficiente e contestuale.

4.2.1 Il Modello e la Strategia di Generazione

Per affrontare il compito cruciale della strutturazione sintattica, è stato impiegato il Large Language Model Gemini 3.5 Flash. A livello di architettura software, il processo è diviso in due step distinti: la generazione di una struttura JSON gerarchica e l'estrazione delle relazioni semantiche tra le entità legali.

4.2.2 Generazione della Struttura Gerarchica

L'efficacia e l'accuratezza del processo di strutturazione si fondano su un *System Prompt* ingegnerizzato ad hoc. Il suo compito principale è quello di mappare il contenuto all'interno del seguente schema logico:

- **Metadati (metadata):** Il modello estrae e categorizza i dati identificativi dell'atto. Vengono isolati campi chiave come la data di stipula, il titolo completo e il nome del file originale.
- **Sezioni Introduttive e Conclusive:** Vengono separate dal corpo normativo tutte le sezioni di contorno. Tra queste figurano le premesse iniziali e le epigrafi (preface), gli indici (preamble), le dichiarazioni di chiusura con le relative firme (conclusions) e gli eventuali documenti annessi (attachments).
- **Corpo del Documento (body):** Rappresenta il nucleo dell'algoritmo di strutturazione. Il modello deve nidificare il testo seguendo fedelmente la gerarchia complessa e rigida tipica degli atti normativi italiani: **Titoli > Capi > Articoli > Commi**.

Per garantire la massima coerenza dei dati all'interno del Knowledge Graph, sono state imposte al LLM regole tassative di parsing:

- **Normalizzazione Numerica:** Tutti gli identificativi gerarchici originariamente espressi in numeri romani o in forma testuale (es. "TITOLO I", "CAPO IV") devono essere categoricamente convertiti in numeri interi arabi (1, 4, ecc.).

- **Gestione Dinamica dei Commi:** Poiché nei testi di legge i commi non sono sempre esplicitamente numerati, il modello è istruito a dedurli analizzando le interruzioni logiche dei paragrafi, assegnando loro una numerazione progressiva.
- **Ottimizzazione dello Schema:** Per mantenere leggeri i file JSON, il modello è programmato per non inizializzare chiavi con valori vuoti o nulli; se un livello gerarchico (come un Capo o un Titolo) non trova riscontro nel testo originale, la proprietà viene semplicemente omessa.

L'esecuzione di questo processo restituisce una directory di documenti JSON puliti e strutturati gerarchicamente, pronti per essere archiviati e per affrontare il successivo step: l'estrazione delle relazioni semantiche tra le entità legali.

4.2.3 Estrazione delle Relazioni Semantiche

La sola scomposizione del testo normativo in una struttura gerarchica ad albero rappresenta una condizione necessaria ma non sufficiente per la costruzione di un vero e proprio *Knowledge Graph*. Per sfruttare appieno la potenza topologica di un database a grafo come Neo4j, è indispensabile individuare e mappare esplicitamente i collegamenti e le dipendenze logiche che intercorrono tra le diverse disposizioni giuridiche all'interno dello stesso atto. A questo scopo, la pipeline prevede un secondo step logico interamente dedicato all'estrazione delle relazioni semantiche. In questa fase, l'input non è più il testo grezzo, ma i file JSON prodotti nello step precedente, che quindi agiscono come una base di conoscenza consolidata. Ciascun documento viene quindi analizzato dal LLM Gemini 3.5 Flash, guidato da un *System Prompt* specializzato. Tale prompt è stato progettato per riconoscere i costrutti linguistici del gergo giuridico che indicano un collegamento formale tra le parti del testo. Le relazioni estratte sono:

- **CITA:** Un articolo o comma fa riferimento a un'altra disposizione.
- **ABROGA:** Una disposizione annulla la validità giuridica di un'altra.
- **DEROGA:** Un elemento introduce un'eccezione a una regola generale.
- **SOSTITUISCE:** Una disposizione prende il posto di un'altra.
- **INTERPRETA:** La comprensione di una norma dipende dalla lettura di un'altra.

L'output di questa analisi è una rete di collegamenti formali, dove per ciascuna relazione vengono specificati il nodo di partenza, il nodo di arrivo e il tipo di legame. Queste informazioni sono esportate in un nuovo set di file JSON.

Questo approccio a due stadi permette di separare le responsabilità e ridurre il carico cognitivo del modello a ogni ciclo. In questo modo si produce un dataset pulito e affidabile, pronto per essere caricato nel grafo.

4.3 Importazione dati nei Database

Una volta completata la strutturazione dei documenti legali in formato JSON, si apre la fase di persistenza e indicizzazione dei dati all'interno dei database. Nel contesto di questo progetto di tirocinio, l'architettura prevede il caricamento dei dati estratti all'interno di due distinte tipologie di database: Neo4j, un database a grafo, e Qdrant, un database vettoriale.

L'uso di due database consente di sperimentare l'uso di entrambi i paradigmi di memorizzazione sullo stesso dominio normativo, al fine di valutarne i rispettivi punti di forza, analizzare le performance e far emergere differenze qualitative e quantitative nelle fasi di interrogazione e navigazione dei testi legali.

Per garantire la coesistenza flessibile di queste due tecnologie e favorire un ambiente di sviluppo pulito e modulare, lo sviluppo del software ha visto l'applicazione combinata di due noti design pattern software[15]: lo *Strategy Pattern* e il *Repository Pattern*.

4.3.1 I Modelli di Dominio

Prima di analizzare le strategie di persistenza, è fondamentale descrivere come i dati estratti vengano rappresentati in memoria. All'interno della cartella `models` di questo step, il sistema definisce le entità fondamentali del dominio normativo utilizzando il paradigma ad oggetti. Queste classi operano essenzialmente come *Entities*: strutture dati pure, prive di logica di persistenza diretta o dipendenze verso i DBMS, il cui unico scopo è incapsulare i dati e preservare i vincoli di integrità del dominio.

L'architettura dei modelli rispecchia fedelmente la scomposizione gerarchica del testo legale:

- **Entity**: Rappresenta l'ente pubblico emittente (ad esempio, un Ministero o un Comune), memorizzando i metadati identificativi dell'istituzione.
- **LegalAct**: Modella l'atto normativo nella sua interezza. Al suo interno, oltre ai metadati generali del documento, espone il metodo `from_json_file(file_path)`, delegato al parsing del file JSON e alla deserializzazione ricorsiva dei suoi elementi strutturali.
- **StructuralElement**: Una classe base polimorfica specializzata nelle sottoclassi `Title`, `Chapter` e `Article`. Permette di mappare i livelli intermedi di aggregazione del testo, mantenendo traccia del numero d'ordine e del titolo della sezione.
- **Paragraph**: Rappresenta il singolo comma contenente il testo, ovvero l'unità atomica d'informazione, sul quale verranno poi calcolati gli embedding.

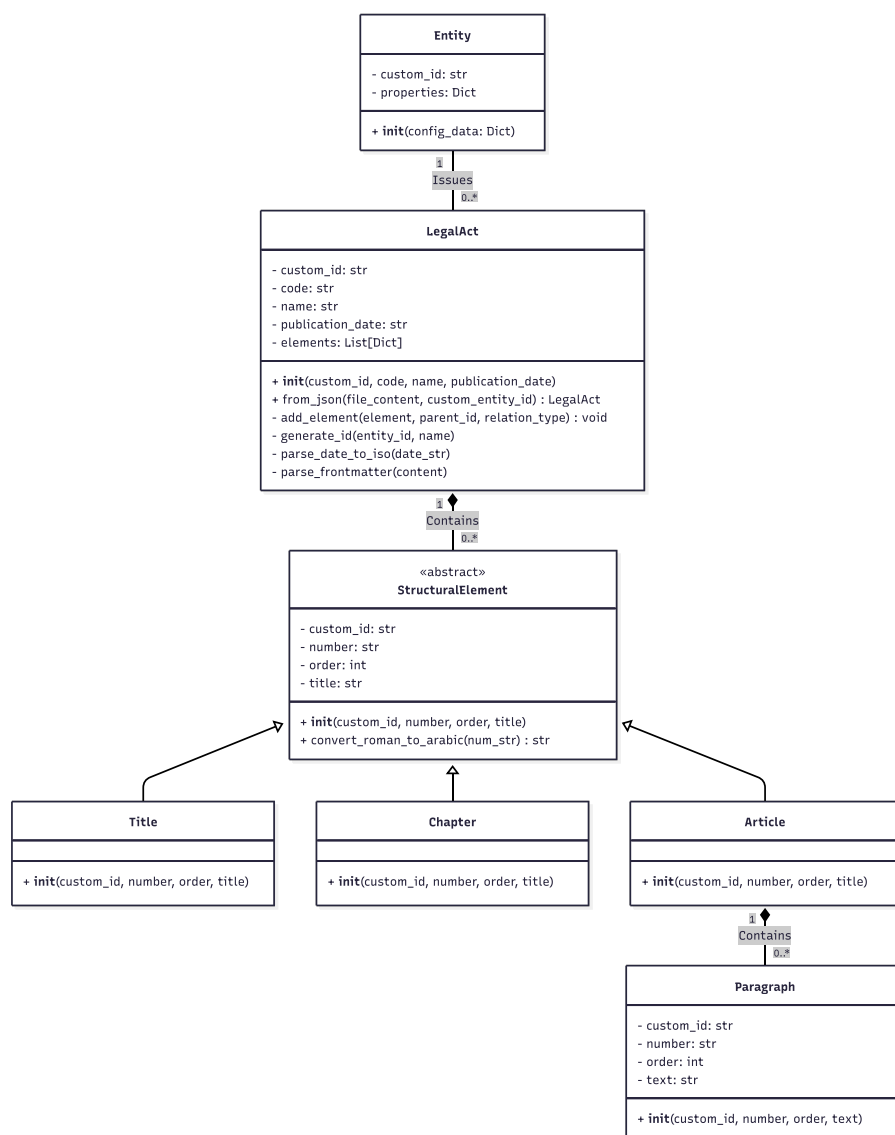


Figura 4.2: Class Diagram dei modelli di dominio

4.3.2 Disaccoppiamento della Persistenza: Lo Strategy Pattern

L'esigenza di effettuare una doppia ingestion prima su Neo4j e poi su Qdrant senza dover duplicare la logica di orchestrazione o modificare il flusso principale del programma ha reso naturale l'adozione del design pattern *Strategy*. Questo pattern comportamentale definisce una famiglia di algoritmi, ne incapsula ciascuno all'interno di una classe e li rende tra loro intercambiabili, consentendo all'algoritmo di variare indipendentemente dai client che lo utilizzano.

Nel sistema sviluppato, l'interfaccia astratta di riferimento è rappresentata dalla classe `DBImporterStrategy`. Essa definisce tre metodi fondamentali che ogni classe concreta

deve implementare:

- `connect()`: per la gestione e la validazione della connessione con l'istanza specifica del database;
- `import_legal_act(entity, legal_act)`: per l'esecuzione materiale della logica di Ingestion dell'atto normativo e dei metadati dell'ente emittente;
- `close()`: per la chiusura sicura delle sessioni.

Nello script principale `main.py`, il ciclo di esecuzione batch istanzia dinamicamente l'oggetto concreto (`Neo4jImporter` o `QdrantImporter`) a seconda della configurazione desiderata. Questa architettura estendibile riduce a zero l'accoppiamento e semplifica future implementazioni con ulteriori database.

4.3.3 Modalità di Accesso ai Dati: Il Repository Pattern

Se lo *Strategy Pattern* isola i database nella loro interezza rispetto all'orchestrazione principale, il *Repository Pattern* interviene specificamente all'interno dell'architettura di persistenza dei database per mediare tra lo strato dei modelli di dominio e lo strato di accesso ai dati fisici. L'obiettivo principale del Repository Pattern è centralizzare la logica di costruzione ed esecuzione delle query, offrendo al client un'interfaccia astratta per poter salvare i dati nei database. In questo modo, l'importer concreto delega l'interazione fisica con il database ai singoli repository, i quali ricevono l'oggetto di dominio da persistere, nascondendo completamente i dettagli delle implementazioni delle query.

4.3.4 Dettagli Implementativi dell'Ingestion su Neo4j

Nel contesto di Neo4j, un atto legale non può essere salvato tramite un'unica operazione atomica standard, a causa delle profonde interconnessioni tra i nodi della gerarchia. Aniché concentrare la scrittura di stringhe Cypher complesse all'interno della classe `Neo4jImporter`, la responsabilità è stata distribuita su una serie di repository specializzati, ognuno deputato alla gestione del ciclo di vita di un singolo modello di dominio:

- `EntityRepository`: Gestisce l'inserimento dei nodi relativi agli enti istituzionali.
- `LegalActRepository`: Inserisce il nodo radice dell'atto e lo ancora all'ente emittente.
- `StructuralElementRepository`: Incapsula la logica Cypher necessaria a creare i nodi intermedi del grafo e a tessere gli archi gerarchici verso i rispettivi nodi padri.
- `ParagraphRepository`: Si occupa esclusivamente della persistenza dei nodi foglia contenenti il testo dei commi.

Per quanto riguarda il salvataggio delle relazioni trasversali (ad esempio, le citazioni tra diversi atti normativi), queste informazioni vengono lette da file JSON esterni dedicati. Il metodo che si occupa di questa operazione attraverso i `custom_id` dei nodi salvati e indicizzati sul grafo trova in maniera quasi istantanea i nodi correlati dall'estratto JSON ed esegue una query Cypher dedicata procedendo alla creazione dell'arco semantico.

Listing 4.1 Esempio di relazione estratta in formato JSON

```

1: {
2:   "sourceId": "ente-provincia-taranto__CCNL-22-02-2010__article
      -3__paragraph-1",
3:   "relationshipType": "CITES",
4:   "targetId": "ente-provincia-taranto__CCNL-22-02-2010__article
      -2"
5: }
```

4.3.5 Dettagli Implementativi dell'Ingestion su Qdrant

La strategia implementata nella classe `QdrantImporter` risponde a logiche radicalmente differenti rispetto a quelle dei database a grafo, dettate dai vincoli operativi dei database vettoriali e dalle necessità algoritmiche della ricerca semantica. Qdrant, infatti, non supporta nativamente strutture gerarchiche ramificate, ma organizza le informazioni in collezioni di vettori piatti denominati *Points*.

Il fulcro del processo di ingestion in Qdrant risiede nell'operazione di appiattimento (*flattening*) della struttura ad albero dell'atto normativo, la quale è stata configurata secondo tre fasi logiche:

- **Identificazione dell'unità fondamentale:** L'unità minima di indicizzazione viene individuata nel modello Paragraph (il singolo comma). Questa porzione di testo rappresenta il compromesso ottimale per bilanciare la granularità informativa e la focalizzazione semantica del modello di embedding.
- **Il problema del contesto assoluto:** Memorizzare un comma in modo isolato causerebbe una perdita critica di contesto, rendendo impossibile determinare a quale articolo o sezione appartenga il testo recuperato.
- **Risoluzione tramite *Tree-Climbing*:** Per preservare l'integrità strutturale, l'algoritmo esegue una procedura di risalita in memoria per ciascun comma. Partendo dall'elemento foglia (il comma), risale ricorsivamente la catena dei nodi padri, estraendo il numero e il titolo di ogni livello gerarchico attraversato (Articolo, Capo, Titolo).

Tutte le informazioni così raccolte vengono infine aggregate e inserite all'interno del dizionario di metadati del punto vettoriale, denominato *Payload*

Listing 4.2 Esempio reale di un Point vettoriale su Qdrant con Payload

```

1: {
```

```

2:   "id": "00020dff-edff-51b8-b2fb-a953ed7e5c0e",
3:   "payload": {
4:     "p_id": "ente-provincia-taranto__regolamento-area-posizioni-
        organizzative-e-alte-professionalita__chapter-2__article-
        -14__paragraph-1",
5:     "text": "Il presente Regolamento entrera in vigore ad
        avvenuta pubblicazione del medesimo all'Albo Pretorio on-
        line nonche sul sito web ufficiale dell'Ente.",
6:     "paragraph_num": "1",
7:     "act_name": "Regolamento Area delle Posizioni Organizzative
        e delle Alte Professionalita.json",
8:     "article_num": "14",
9:     "article_name": "Entrata in vigore",
10:    "chapter_num": "2",
11:    "chapter_name": "Area delle Alte Professionalita"
12:  },
13:  "vector": [/* Rappresentazione numerica embedding testuale */]
14: }

```

4.3.6 Risultati e Metriche

Al termine delle procedure di ingestion sui documenti dell'Ente Provincia di Taranto, è stato possibile raccogliere i dati quantitativi relativi ai tempi di esecuzione e ai volumi di record inseriti per entrambi i database.

I log di sistema hanno registrato le metriche riassunte nella Tabella 4.1.

Database	Totale Elementi	Commi Inseriti	Relazioni	Tempo (Ingestion)
Neo4j (Grafo)	3720 Nodi	2763	4176	18.53 s
Qdrant (Vettoriale)	2763 Punti	2763	N/D (Flat)	0.34 s

Tabella 4.1: Confronto metriche e tempi tra Neo4j e Qdrant.

I risultati mettono in luce in modo evidente le notevoli differenze operative tra i due sistemi in fase di scrittura. La costruzione topologica di Neo4j richiede tempi di elaborazione maggiori a causa dei molteplici controlli sui vincoli di unicità e della creazione delle innumerevoli relazioni tra i nodi, mentre l'approccio "piatto" di Qdrant si dimostra estremamente rapido per il caricamento massivo dei vettori.

4.4 Creazione degli Embedding

L'operazione di vettorizzazione, o di *embedding*, rappresenta un passaggio cruciale per abilitare le funzionalità di ricerca semantica sui documenti legali all'interno del sistema.

4.4.1 Astrazione e Gestione dei Modelli di Embedding

Per garantire la massima flessibilità nell'uso di modelli eterogenei, è stata definita un'interfaccia astratta base denominata `EmbeddingModel`. Da essa derivano le implementazioni specifiche per le diverse tipologie di erogazione dei modelli:

- **Modelli Locali (`LocalEmbeddingModel`):** consentono l'esecuzione di modelli open-source direttamente in locale sfruttando la libreria `SentenceTransformer`[16]. I modelli vengono salvati in una cache locale per ottimizzare i caricamenti successivi.
- **Modelli Cloud HuggingFace (`HuggingFaceCloudModel`):** si interfacciano con la *HuggingFace Inference API*[17]. Per garantire la resilienza del sistema, la classe implementa un meccanismo di tolleranza ai guasti basato su tentativi multipli in caso di errore di rete o di superamento dei limiti di utilizzo, con attese esponenziali tra i tentativi.
- **Modelli Cloud Gemini (`GeminiCloudModel`):** comunicano tramite l'endpoint REST `embedContent` di Google. Anche qui è stato implementato il meccanismo di retry e, inoltre, per ottimizzare ulteriormente il throughput, viene utilizzata una strategia di rotazione circolare su un pool di API keys fornite in configurazione.

In particolare, sono stati integrati quattro specifici modelli di embedding per verificare le differenze prestazionali e di accuratezza in fase di retrieval:

- **google/embeddinggemma-300m:** Si tratta dell'unico modello della suite eseguito interamente in locale sfruttando le risorse hardware della macchina tramite la libreria `SentenceTransformer`. Produce vettori con una dimensionalità di 768.
- **intfloat/multilingual-e5-large:** Modello ad alte prestazioni fruibile in cloud tramite la *HuggingFace Inference API*. Caratterizzato da una dimensionalità di 1024 vettori, questo modello richiede obbligatoriamente un pre-processing del testo tramite prefissi asimmetrici per ottimizzare le prestazioni di recupero: nello specifico, viene anteposto il token "passage: " per i testi da memorizzare e il token "query: " per le stringhe di ricerca.
- **BAAI/bge-m3:** Modello cloud multilingua integrato anch'esso tramite l'infrastruttura di *HuggingFace*. Genera vettori a 1024 dimensioni ed è particolarmente rinomato per la sua versatilità nel mappare relazioni semantiche complesse in contesti cross-lingua e multi-lingua.
- **gemini/gemini-embedding-001:** Modello cloud proprietario di Google accessibile mediante l'endpoint REST `embedContent`. Configurato con vettori a 768 dimensioni, è nativamente ottimizzato per task di *text retrieval* su larga scala.

4.4.2 Integrazione con i Database

La distribuzione dei vettori generati ai diversi database viene gestita, anche qui, attraverso lo *Strategy Pattern*, che isola le logiche di inserimento dei dati per ciascun database.

Un aspetto cruciale condiviso da entrambe le strategie è l'**arricchimento contestuale del testo**. Al fine di massimizzare l'accuratezza della ricerca semantica, il sistema non si limita a generare l'embedding del solo testo grezzo del comma. Al contrario, il testo viene dinamicamente concatenato con i suoi metadati gerarchici e descrittivi (come il nome dell'atto normativo, il numero e il titolo del capitolo, e il numero e la rubrica dell'articolo).

La Strategia Neo4j

La `Neo4jEmbeddingStrategy` gestisce l'estrazione, la vettorizzazione e il salvataggio degli embedding nel database a grafo. Il processo si articola in tre fasi principali:

1. **Estrazione e arricchimento contestuale:** Il sistema esegue una query Cypher che risale la gerarchia del documento a partire dal testo del nodo (`:Paragraph`). La query estrae i metadati dei nodi padre (Atto, Titolo, Capo, Articolo) e li concatena al contenuto del comma, generando una stringa arricchita e contestualizzata.
2. **Generazione degli embedding:** Il testo completo viene inviato ai modelli di *embedding* che processano e traducono il testo in vettori numerici in grado di catturarne il significato semantico.
3. **Salvataggio nel database:** I vettori generati vengono salvati come proprietà del nodo (`:Paragraph`) originale. La proprietà del nodo è dinamica e specifica per il modello utilizzato (es. `embedding_<model-name>`), questo per garantirne l'accessibilità e la tracciabilità.

La Strategia Qdrant

La classe `QdrantEmbeddingStrategy` verifica a runtime lo schema della collezione, inizializzandone, ove necessario, la configurazione per i *Named Vectors*. Tale approccio consente di associare a un singolo record vettoriale molteplici rappresentazioni prodotte dai modelli differenti, agevolando le prestazioni in fase di retrieval.

In linea con l'approccio adottato per Neo4j, il processo implementa un pre-processing di arricchimento semantico. Prima della generazione dell'embedding, vengono estratti i metadati dal *payload* del database per costruire una gerarchia di contesto testuale (es. "Atto: Codice Civile | Titolo 1 | Capo 2 | Art. 4").

–5–

Ricerca Semantica

CONTENTS: 5.1 Introduzione alla Ricerca Semantica. 5.2 Setup Sperimentale. 5.3 Metriche di valutazione. 5.4 Risultati Sperimentali. 5.5 Analisi Comparativa e Conclusioni. 5.6 Embedding di Neo4j con relazioni.

5.1 Introduzione alla Ricerca Semantica

La capacità di recuperare in modo preciso e tempestivo le normative pertinenti a partire da query in linguaggio naturale rappresenta un requisito fondamentale per l'efficacia del progetto proposto. Con il termine "ricerca vettoriale" ci si riferisce a una tecnica che permette di recuperare informazioni in base al loro significato semantico piuttosto che alla corrispondenza esatta delle parole chiave. Questo processo prevede che sia i testi archiviati che le domande degli utenti vengano tradotti in vettori numerici. La pertinenza tra una query e un testo viene calcolata tramite la *Cosine Similarity* (Similarità del Coseno), una metrica che misura l'ampiezza dell'angolo tra due vettori in uno spazio multidimensionale: minore è l'angolo, maggiore è l'affinità semantica tra i due contenuti.

Questo capitolo illustra le performance della *Semantic Search* sui documenti legali precedentemente salvati. L'obiettivo del benchmarking è valutare le prestazioni della *vector-search* applicata al dominio legale, analizzando il comportamento del sistema al variare dei parametri di ricerca e proponendo un confronto prestazionale approfondito tra i molteplici modelli di embedding (descritti nella Sezione 4.4) e i due database vettoriali adottati (Neo4j e Qdrant, introdotti nella Sezione 4.3).

5.2 Setup Sperimentale

Nel contesto di questa valutazione, il set di test comprende 70 domande in linguaggio naturale, ciascuna delle quali ammette come risposta corretta un singolo e specifico

comma, all'interno di un dominio normativo composto da 44 documenti, per un totale di 2763 commi.

5.2.1 Definizione della soglia di ricerca: il parametro Top-K

La capacità di recupero del sistema è stata analizzata variando la soglia dei risultati restituiti, definita dal parametro Top-K. Nell'ambito dell'*Information Retrieval*, il parametro K indica il numero di documenti (nel nostro dominio, i commi) maggiormente pertinenti che il motore di ricerca seleziona per rispondere a una query. Operativamente, dopo aver calcolato la *Cosine Similarity* tra il vettore della query e quelli di tutti i documenti del sistema, i risultati vengono ordinati in modo decrescente in base al punteggio ottenuto. Selezionare i Top-K risultati significa estrarre esclusivamente i primi K elementi di questa graduatoria, ovvero quelli semanticamente più affini alla domanda.

Ai fini della sperimentazione, sono state impostate soglie di Top-K pari a 10, 5, 3 e 1. Tale approccio a soglie decrescenti ha lo scopo di misurare con quale frequenza percentuale il singolo comma atteso rientri nelle prime K posizioni del ranking. In uno scenario ideale, il sistema dovrebbe essere in grado di posizionare la risposta corretta sempre al primo posto (Top-1), tuttavia, analizzare soglie più ampie permette di valutare se il sistema riesce comunque a confinare l'informazione corretta all'interno di un sottoinsieme ristretto di commi.

5.2.2 Allineamento delle strategie di ricerca: ANN vs Exact k-NN

Durante i test preliminari per l'esecuzione della ricerca, è emersa una differenza sostanziale nel comportamento predefinito dei due database. Neo4j applicava di default algoritmi di ricerca approssimata (*Approximate Nearest Neighbors* o ANN)[18], i quali privilegiano la velocità ma introducono un margine di errore che ha portato, in alcuni casi, alla mancata estrazione di commi pertinenti. Al contrario Qdrant, essendo un database vettoriale puro, adottava nativamente una strategia di scansione esatta per dataset di dimensioni contenute (inferiori ai 20.000 record).

Al fine di garantire un confronto prestazionale equo tra i modelli e tra le due soluzioni di storage, la configurazione di ricerca è stata uniformata forzando l'esecuzione in modalità di ricerca esaustiva (*Exact k-NN*) su entrambi i database. Questa scelta ha permesso di misurare con assoluta precisione matematica il piazzamento delle risposte, annullando le discrepanze che sarebbero state introdotte dagli algoritmi di approssimazione.

5.3 Metriche di valutazione

L'analisi si focalizza su diverse metriche chiave utilizzate nei task di Information Retrieval:

- **HIT@K**: La percentuale di query per cui il comma corretto è presente nei primi K risultati restituiti dal database.

- **Latenza Media della Ricerca:** Il tempo medio impiegato dal database per elaborare la similarità semantica e restituire i risultati vettoriali, espresso in millisecondi (ms).

5.4 Risultati Sperimentali

5.4.1 Precisione a Confronto (HIT@K)

Dai risultati emerge un dato fondamentale: **la precisione della ricerca (HIT@K) è IDENTICA su entrambi i database per ogni modello di embedding e per ogni valore di K.**

Entrambi i database archiviano e recuperano esattamente gli stessi vettori generati dagli stessi modelli. Se configurati correttamente per eseguire una ricerca esaustiva basata sulla *Cosine Similarity*, l'ordine dei risultati restituiti (e quindi la precisione rispetto al *ground truth*) deve necessariamente essere uguale.

Modello di Embedding	HIT@1	HIT@3	HIT@5	HIT@10
google/embeddinggemma-300m	58.6%	92.9%	95.7%	97.1%
intfloat/multilingual-e5-large	55.7%	88.6%	91.4%	94.3%
BAAI/bge-m3	55.7%	88.6%	95.7%	97.1%
gemini/gemini-embedding-001	67.1%	91.4%	92.9%	98.6%

Tabella 5.1: Risultati di Precisione (identici per Neo4j e Qdrant)

L'analisi dei modelli sulla base della precisione evidenzia che:

- **Migliore per singola risposta (HIT@1):** gemini/gemini-embedding-001 vince nettamente (67.1%) se l'obiettivo è avere il risultato esatto al primissimo posto.
- **Miglior compromesso (Top 3-5):** google/embeddinggemma-300m offre l'equilibrio ideale per i motori di ricerca, superando tutti nella fascia dei primi 3-5 risultati (92.9% - 95.7%).
- **Finestre ampie:** BAAI/bge-m3 è ottimo per recuperare contesto esteso (HIT@10), mentre intfloat/multilingual-e5-large risulta in generale il più debole su questo dataset.

5.4.2 Analisi dei Tempi e Latenze

Sebbene i risultati di precisione siano identici, le prestazioni in termini di latenza mostrano differenze sostanziali.

Modello di Embedding	Neo4j (Media)	Qdrant (Media)
google/embeddinggemma-300m	109.70 ms	4.40 ms
intfloat/multilingual-e5-large	184.03 ms	7.02 ms
BAAI/bge-m3	181.61 ms	6.74 ms
gemini/gemini-embedding-001	107.16 ms	6.45 ms

Tabella 5.2: Latenza Media Aggregata di Ricerca Vettoriale

Come si evince dalla Tabella 5.2, **Qdrant è mediamente dalle 15 alle 30 volte più veloce di Neo4j** nell'eseguire le singole query vettoriali. I tempi di Qdrant si attestano quasi sempre sotto i 10 ms, mentre quelli di Neo4j superano regolarmente i 100-180 ms.

Focalizzando l'analisi sui singoli modelli, è degno di nota come quelli che producono vettori a 768 dimensioni (embeddinggemma-300m e gemini-embedding-001) risultino costantemente più veloci, su entrambi i database, rispetto a quelli a 1024 dimensioni (bge-m3 e multilingual-e5-large). Ciò evidenzia come la complessità computazionale del calcolo vettoriale scali proporzionalmente alla dimensionalità dell'embedding.

La marcata differenza di latenza generale si spiega analizzando la natura architetturale dei due database:

1. **Qdrant:** È un database vettoriale nativo, ottimizzato per l'indicizzazione e la ricerca su spazi vettoriali ad alta dimensionalità. Essendo sviluppato in Rust, beneficia di un controllo diretto della memoria che si traduce in latenze estremamente basse, overhead minimizzato e un'ottimizzazione spinta per il calcolo matematico e il parallelismo hardware.
2. **Neo4j:** Nasce come database a grafo per la gestione di relazioni complesse, a cui la ricerca vettoriale è stata integrata successivamente. Essendo sviluppato in Java, ogni operazione sconta inevitabilmente l'overhead della JVM, i cicli del *Garbage Collector* e la necessità di parsare le richieste attraverso il motore delle query Cypher, fattori che innalzano la latenza di base rispetto a un'architettura in Rust.

5.5 Analisi Comparativa e Conclusioni

In base a quanto emerso, l'infrastruttura ideale dipenderà dal bilanciamento tra necessità *Real-Time* nella ricerca per similarità (dove Qdrant eccelle) e necessità di ricostruzione topologica del documento (dove Neo4j è essenziale e offre maggiore stabilità come vedremo nella prossima sezione).

I modelli google/embeddinggemma-300m e gemini/gemini-embedding-001 si confermano tra le scelte tecnologiche più robuste per l'elaborazione del linguaggio della Pubblica Amministrazione italiana.

5.6 Embedding di Neo4j con relazioni

Entrambi i database testati si comportano come "recipienti" per vettori generati esternamente e basati sul solo testo. Essendo Neo4j progettato per i grafi, possiede algoritmi interni (*Graph Native Machine Learning*) in grado di generare vettori che tengono conto anche delle *relazioni*. Considerando le relazioni già mappate (CITA, INTERPRETA, ABROGA, SOSTITUISCE, DEROGA), arricchire la ricerca vettoriale con il contesto topologico della rete normativa potrebbe incrementare significativamente l'accuratezza del sistema.

Bibliografia

- [1] *Neo4j Graph Platform*. URL: <https://neo4j.com>.
- [2] *Qdrant Vector Database*. URL: <https://qdrant.tech>.
- [3] *Python Language Reference*. URL: <https://www.python.org>.
- [4] *Open Data Loader*. URL: <https://github.com/opendataloader-project/opendataloader-pdf>.
- [5] *Docling: Document Parsing and Layout Analysis*. URL: <https://github.com/DS4SD/docling>.
- [6] *RapidOCR: OCR Engine*. URL: <https://github.com/RapidAI/RapidOCR>.
- [7] *Cypher Query Language Reference*. URL: <https://neo4j.com/docs/cypher-manual>.
- [8] *Gemini 3.5 Flash Model Card*. URL: <https://storage.googleapis.com/deepmind-media/Model-Cards/Gemini-3-5-Flash-Model-Card.pdf>.
- [9] *Multilingual E5 Large Embedding Model*. URL: <https://huggingface.co/intfloat/multilingual-e5-large>.
- [10] *Gemini Embedding Model (gemini-embedding-001)*. URL: <https://ai.google.dev/gemini-api/docs/embeddings>.
- [11] *EmbeddingGemma-300M: A Compact Embedding Model*. URL: <https://huggingface.co/google/embeddinggemma-300m>.
- [12] *BGE-M3: Multilingual Embedding Model*. URL: <https://huggingface.co/BAAI/bge-m3>.
- [13] *Docker: Containerization Platform*. URL: <https://www.docker.com>.
- [14] *Document management — Portable document format — Part 1: PDF 1.7*. Standard. Sezione 14.8: Tagged PDF. Geneva, Switzerland: International Organization for Standardization, 2008. URL: <https://www.iso.org/standard/51502.html>.
- [15] Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. ISBN: 978-0201633610.
- [16] *Sentence Transformers: Multilingual Sentence Embeddings*. URL: <https://www.sbert.net>.

-
- [17] *HuggingFace Inference API*. URL: <https://huggingface.co/docs/api-inference>.
 - [18] Jingdong Wang, Ting Zhang, Jingkuan Song e Heng Tao Shen. *Efficient Vector Search: Approximate vs Exact Nearest Neighbors*. 2021. URL: <https://arxiv.org/abs/2106.03478>.